

GOVERNANCE PLANE · V0.1

Agent *Sentinel*

The control plane that gates every AI agent tool call, signs the audit trail, and meters per-BU spend.

Built on Gemini 2.5 Flash + Pro with Cached Content over full policy documents.

TRACK 2 · GOOGLE AI STUDIO

TRACK 1 · SECURITY & GOVERNANCE

SANKAR SUBBAYYA · 2026.05.19 · AI & BIG DATA EXPO · SAN JOSE

The three open questions every CIO has

COMPLIANCE OFFICER

"Who approved this action, and against which policy?"

Today: logs at best. No cited evidence.

CISO

"How do I stop my agent from doing the wrong thing?"

Today: prompt engineering. No enforcement layer.

CFO

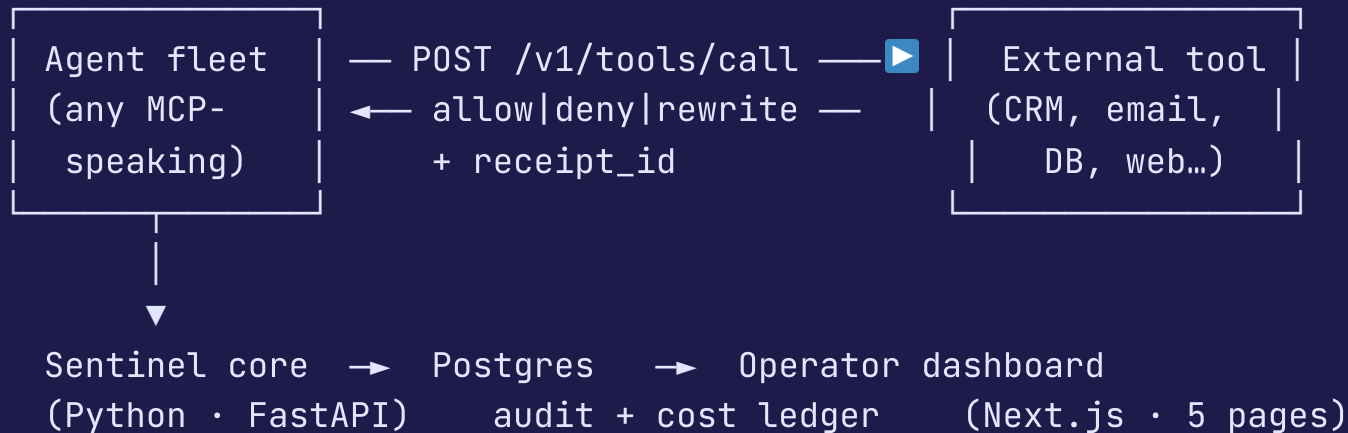
"What is our agent fleet costing each business unit?"

Today: aggregated bill. No attribution.

>70% of F500s have agentic pilots in 2026. <10% are in production. The bottleneck is not model quality — it is operational governance.

What Sentinel does

A single FastAPI process sits between any MCP-speaking agent fleet and the tools they call. Every action flows through a four-stage decision pipeline, becomes a signed audit receipt, and emits a per-business-unit cost event.



Drop-in. Any agent that already speaks MCP `tools/call` is a client — no code changes. **Survives demo conditions.** Runs end-to-end without a Gemini API key via deterministic stub fallbacks.

The four-stage gating pipeline

01

Static engine

<5 ms

Regex denylists, role ACL, refund cap, plaintext PII. No LLM tokens burned on obvious cases.

02

Drift detector

~0 ms

Injection markers in args, tool-vs-declared-goal mismatch, fresh external recipients.

03

Flash gate

<100 ms p95

Gemini 2.5 Flash · `response_schema` · `thinking_budget=0` . Returns GateDecision JSON.

04

Pro escalation

<2 s, ~3-5% of calls

Gemini 2.5 Pro · `cached_content` over whole policy docs. Cited rationale.

Each decision is **hash-chained, HMAC-signed**, and **cites the exact policy version used** — tamper-evident without a blockchain.

Why Gemini — the sponsor-native story

GEMINI 2.5 FLASH · THE GATE

Structured output via

```
response_schema=GateDecision .  
thinking_budget=0
```

for sub-100ms
p95. Every tool call hits Flash.

GEMINI 2.5 PRO · THE REASONER

1M-context window means Pro

consumes **whole policy documents** per
call. **No chunking. No RAG drift.** Cited
rationale.

CACHED CONTENT · THE ECONOMICS

~75% token cost savings on stable
policy bundles. Refreshed every 6h.
Per-call governance becomes
economically viable.

FILES API · THE POLICY LIBRARY

Authoritative policy storage. Multimodal
extraction via Flash. PolicyPipe is 5
modules, ~600 LOC. No LangChain
wrapper hiding the SDK.

This combination — Flash structured output + Pro long-context + Cached Content + Files API — is **uniquely possible on Gemini**.

Live demo (2:30)

t	Beat	What you see
0:00	Setup · three Gemini agents wired through Sentinel	Sales · Finance · Customer Ops live tiles
0:20	Happy path · Sales pulls competitor pricing	• ALLOW · 1 ms · \$0.00065 Sales BU
0:50	Policy violation · Finance emails PII to a vendor	↻ REWRITE · body redacted · cites <i>Data Handling v3.2</i>
1:30	Red team · prompt injection in refund memo	× DENY · drift signal cited · Pro upholds
1:55	Compliance pull · filter receipts by agent + decision	cited timeline with policy chips
2:15	CFO view · BU spend rollup	stacked bar · base vs Gemini · per-BU \$ attribution

Bonus — Brief mode. Type a natural-language brief at `/agent` (or `sentinel agent run`) — Gemini Flash function-calls turn-by-turn while Sentinel gates every call. Same audit + cost ledger as the direct API.

What's defensible — and measured

96.8%

EVAL ACCURACY

150 / 155 cases pass across 12 categories — happy path, PII variants, injection variants, role matrix, regulatory, edge cases, drift. `sentinel eval run`.

3 ms

LATENCY P99

Across 155 cases. p50 0 ms · p95 2 ms. Static engine handles 34% in <1 ms; Flash 47% in <5 ms; Pro 20% in <10 ms.

BROKEN

TAMPER DETECTED

Per-agent `prev_hash → self_hash` + HMAC. `sentinel ledger verify` caught a single-byte mutation in a stored rationale and marked the chain BROKEN. Exit code 1.

~600

POLICYPIPE LOC

Five Python modules. No chunking, no vector DB, no LangChain. Direct Gemini Files API + Cached Content. Visible to sponsor judges.

Three buyers, one architecture

Stakeholder	Their pain	Their surface	Evidence in the receipt
Compliance Officer	"Prove enforcement happened in real time."	<code>/receipts</code> · filterable cited timeline	<code>policy_versions_used</code> , <code>self_hash</code> , <code>signature</code>
CISO	"Stop indirect prompt injection."	<code>/redteam</code> · synthetic adversarial calls	<code>escalated=true</code> , <code>[drift:<reason>]</code> in rationale
CFO	"Charge agent spend back to BUs."	<code>/cost</code> · stacked \$ per BU, 1/7/30-day window	<code>cost_event {bu, base, gemini, total}</code>

Three slides for three buyers in the deck of every CIO meeting we'll walk into. **One Gemini-native architecture serves all of them.**

Phase 2 · shipped tonight

NATIVE ADAPTER: GOOGLE ADK (FLAGSHIP)

Three-line wrap of any ADK `FunctionTool` or whole `Agent`. Every tool the agent invokes is gated, audited, costed. Plus Anthropic Agent SDK, OpenAI tool-calling, CrewAI, generic MCP.

CUSTOMER KMS · ROTATED KEYS

Per-receipt `key_id`; verifier supports historical keys. Rotate via env: add a new key to `SENTINEL_SIGNING_KEYS`, bump `SENTINEL_ACTIVE_KEY_ID`, restart. Old receipts continue to verify.

ON-CHAIN RECEIPT ANCHORING

SHA-256 Merkle root over unanchored receipts → `anchor_batches` table. Targets: local file, OpenTimestamps, Circle Arc state channel. **Connects to our Arc/Circle nanopayment work.**

SLACK / TEAMS ALERTS · OBSERVE MODE · POLICY AUTHORIZING

Webhook alerts on every deny + rewrite. `POST /v1/observe` for read-only deployments. `POST /v1/policies/text` for inline policy authoring. Multi-policy conflict detection on Pro escalations.

QUICKSTART · < 90 SECONDS

Try it *now*.

Cloudflare for AI agents — built on Gemini.

All six demo beats verified end-to-end. Stub-mode fallback so a missing API key never blocks the demo.

```
git clone https://github.com/SankarSubbayya/agent_sentinel
cd agent_sentinel && createdb agent_sentinel
uv sync && uv run sentinel init-db
uv run sentinel serve --port 8088 # gateway
cd dashboard && npm i && PORT=3030 npm run dev # dashboard
uv run sentinel demo run # 2:30 walkthrough
```